

第四章 字符串

1.求下列字符串的next数组

```
t='abcdaabccddaba' s='XYXYXZXZXYXZ'
```

```
t_next=[-1,0,0,0,0,1,1,2,3,4,0,1,2] s_next=[-1,0,0,0,1,2,0,1,0,1,2,3,4]
```

2.请设计算法消除字符串中的b和ac，需要满足条件

(1)字符串只能遍历一次
(2)不能够使用额外空间，即 $O(1)$ 的空间
如"acbac" -> " ", "ababac" -> "aa"

1. 使用一个指针/数组索引 w 指向当前写入的位置，另一个指针/索引 r 用于遍历字符串。

- 遍历字符串，如果遇到 "b"，直接跳过；如果遇到 "a"，检查下一个字符：
- 如果下一个字符是 "c"，则跳过这两个字符。
- 否则，将 "a" 写入当前位置。

2. 将 write 指针向前移动。

3. 最后，处理指针的位置，将剩余字符移到字符串的开始部分。

2.

求已知长度为 m 的字符串 P 和长度为 n 的字符串 T ， $n > m$ ， $S = PT$ (由 P 和 T 连接形成的长度为 $m+n$ 的字符串)，类似于next数组，在 S 上定义next数组set数组，定义 $nextset[i]$ 如下：

- 分两种情况，说明如何通过nextset数组来判断 P 是否在文本 T 中出现
- (1)若 P 不会出现在 T 尾部，即 $P \dots$ ，写出充要条件并证明
- (2)若 P 可能出现在 T 尾部，写出充要条件单不需证明

(1) 充要条件 $nextset$ 数组最后一位 $< m$

证明

- (充分性)由nextset定义可知，nextset的最后一位 $< m$ ，就说明 S 最后可以和前面的 P 匹配的子串长度小于 m ，而 P 的长度为 m ，说明 P 不会出现在 T 尾部
- (必要性)同样由nextset数组定义可知，如果 P 不会出现在 T 尾部，next数组最后一位必然 $< m$

(2)若 P 可能出现在 T 尾部，写出充要条件但不需要证明

$nextset$ 数组最后一位 $= m$

3.

求给定一个长度为 n 的字符串 S ，和一个整数 i ，要求设计一个线性时间，常数空间的算法，将 $S[0 \dots i]$ 移动到 $S[(n - i - 1) \dots (n - 1)]$ ，将 $S[(i + 1) \dots (n - 1)]$ 移动到 $S[0 \dots (n - i - 2)]$ 。如输入"HELLOWORLD"， $i=4$ ，输出"WORLDHELLO"。

```
void reverse(char* str, int start, int end) {
    while (start < end) {
        std::swap(str[start], str[end]);
        start++;
        end--;
    }
}

void rotateString(char* str, int n, int i) {
    // 反转 S[0...i]
    reverse(str, 0, i);

    // 反转 S[i+1...n-1]
    reverse(str, i + 1, n - 1);

    // 反转整个字符串
    reverse(str, 0, n - 1);
}
```

调用rotateString(S,n,i)即可实现